The University of Central Punjab
Department of Computer Science,
Object-Oriented Programming
(OOP)

Assignment: Advanced OOP Programming

Submission Guidelines for GitHub Classroom:

Course: CSCP2023 – Object-Oriented Programming | **Total Marks:** 100

Student Name: _____ **Reg. No.:** _____ **Date:** _____

1. Submit the complete assignment through the assigned GitHub Classroom repository.
2. Use separate header (`.h`) and implementation (`.cpp`) files for each class.
3. Include a `main.cpp` file to demonstrate all implemented functionality.
4. Use meaningful variable and function names.
5. Implement appropriate constructors and destructors.
6. Do not use global variables.
7. Do not make private data members public.
8. Ensure that dynamically allocated memory is properly released.
9. Test all overloaded operators with multiple objects.
10. Include a `README.md` explaining the project structure and compilation instructions.

Important Restrictions

- Do not use inheritance.
- Do not use STL containers to replace the required arrays and dynamic memory implementation.
- Do not use shallow copying where deep copying is required.
- Do not leave memory leaks or dangling pointers.
- Operator overloads must follow logical and consistent behavior.
- Each student must submit their own implementation.

Evaluation Criteria

Component	Marks
Question 1: Student Management System	20
Question 2: Digital Wallet System	25
Question 3: Inventory Management System	25
Question 4: Hospital Management System	30
Total	**100**

Note: Students may be asked to demonstrate their programs and explain their implementation during evaluation.

Course Instructor: Qazi Ammar Arshad

Question 1: University Student Registration and Performance System (20 Marks)

Design a C++ program to manage student registrations and academic performance at a university. Each student should have attributes such as student ID, name, semester, and CGPA. The system should allow the university to store records of multiple students and perform operations on them.

Create a `Student` class with an array of objects to store student records. The class should also maintain a static variable to keep track of the total number of students registered in the system. The student ID must be constant and initialized through a constructor initializer list.

The program should allow users to enter details for multiple students and search for a student using their registration ID. Implement a function that accepts a pointer to a Student object to display or update its information. Use the `this` pointer in appropriate member functions to resolve naming conflicts.

Additionally, implement a const member function to display student details without modifying the object. Provide a static function to display the total number of registered students.

Sample Output:

```
Enter the number of students: 3

Enter details for Student 1:
Student ID: 101
Name: Ali
Semester: 6
CGPA: 3.45

Enter details for Student 2:
Student ID: 102
Name: Sara
Semester: 7
CGPA: 3.82

Enter details for Student 3:
Student ID: 103
Name: Ahmed
Semester: 5
CGPA: 3.20

Enter Student ID to search: 102

Student Record Found:
Student ID: 102
Name: Sara
Semester: 7
CGPA: 3.82

Total Registered Students: 3
```

Question 2: Digital Wallet and Transaction Management System (25 Marks)

A financial technology company wants to develop a Digital Wallet Management System that allows users to maintain their wallet balances and transaction histories.

Create a `Wallet` class containing the following attributes:

- Wallet ID
- Owner Name
- Balance
- Dynamically allocated array to store transaction amounts
- Number of transactions

The wallet ID must be a constant data member. The system should maintain a static variable to track the total number of wallets created.

The program should allow users to deposit money, withdraw money, and record each transaction in the wallet's transaction history. Each wallet should have its own independent transaction history.

Since transaction records are stored dynamically, implement a copy constructor that performs a deep copy. When a wallet is copied, modifying or deleting the copied wallet must not affect the original wallet.

Overload the following operators:

1. `+` to combine the balances of two wallets and return the resulting amount or a new wallet according to your design.
2. `-` to calculate the difference between two wallet balances.
3. `==` to check whether two wallets have equal balances.
4. `=` to assign one wallet's contents to another while handling dynamically allocated memory safely.
5. `<<` to display wallet information and transaction history.

Implement a friend function named `compareWallets()` that accesses the private balance of two wallets and identifies which wallet has a higher balance.

Sample Output:

```
Enter details for Wallet 1:
Wallet ID: 501
Owner Name: Ali
Initial Balance: 10000

Enter details for Wallet 2:
Wallet ID: 502
Owner Name: Sara
Initial Balance: 15000

Wallet 1 Transactions:
Deposit: 2000
Withdraw: 1000

Wallet 2 Transactions:
Deposit: 3000
Withdraw: 500
```

```
Wallet 1 Balance: Rs. 11000
Wallet 2 Balance: Rs. 17500

Comparing Wallets:
Wallet 2 has a higher balance.

Combining Wallet Balances:
Combined Balance: Rs. 28500

Checking Equality:
Wallets do not have equal balances.
```

Additional Requirement:

Create a copy of Wallet 1 and modify its transaction history. Display both the original and copied wallet records to demonstrate that changes to the copied wallet do not affect the original.

Question 3: E-Commerce Inventory and Product Management System (25 Marks)

An online shopping company wants to develop an inventory management system that stores and manages different products available for purchase.

Create a `Product` class containing:

- Product ID
- Product Name
- Category
- Price
- Quantity in Stock

The product ID should be constant and initialized using a constructor initializer list. Maintain a static variable to count the total number of products stored in the inventory.

The system should allow users to enter details for multiple products using an array of objects. Users should be able to search for products based on their IDs and display products within a specified price range.

Implement a `ResourceManager` class that acts as an inventory administrator. Declare it as a friend class of `Product` so it can directly access private product information to update stock quantities and display inventory details.

Overload the following operators in the `Product` class:

1. `+` to combine the stock quantities of two products having the same product ID.
2. `==` to check whether two products have the same product ID.
3. `>` to compare two products based on their prices.
4. `++` (prefix and postfix) to increase the stock quantity of a product by one.
5. `[]` to access a product's transaction or stock history stored in a dynamically allocated array, if implemented as part of your design.
6. `<<` to display product details.

The program should also include a function that receives a pointer to a Product object and updates its stock quantity.

Sample Data:

Product ID	Product Name	Category	Price (Rs.)	Quantity
101	Laptop	Electronics	100000	10
102	T-Shirt	Clothing	850	25
103	C++ Book	Books	500	40
104	Mouse	Electronics	1200	15

Sample Output:

```
Enter minimum price: 300
Enter maximum price: 1500

Products within the specified price range:
```

```
Product ID: 102
Name: T-Shirt
Category: Clothing
Price: Rs. 850
Quantity: 25

Product ID: 103
Name: C++ Book
Category: Books
Price: Rs. 500
Quantity: 40

Product ID: 104
Name: Mouse
Category: Electronics
Price: Rs. 1200
Quantity: 15

Updating Stock:
Enter Product ID: 103
Enter quantity to add: 5

Updated Stock:
Product: C++ Book
Quantity: 45

Total Products: 4
```

Question 4: Hospital Billing and Patient Record Management System (30 Marks)

A hospital requires a computerized system to manage patient records and calculate treatment bills. Each patient should have a patient ID, name, age, and a dynamically allocated array storing the daily treatment charges.

Create a `Patient` class with the required attributes. The patient ID should be constant and initialized through a constructor initializer list. The class should also contain a static variable to track the total number of patients registered.

The hospital should allow users to enter records for multiple patients, calculate their total treatment costs, and display complete patient histories.

Implement the following requirements:

A. Dynamic Memory Management

- Store daily treatment charges in a dynamically allocated array.
- Implement a parameterized constructor.
- Implement a copy constructor using deep copying.
- Implement a destructor to release dynamically allocated memory.
- Overload the assignment operator to safely copy one patient's record into another.

B. Friend Function and Friend Class

- Implement a friend function named `compareBills()` that compares the total bills of two patients and displays the patient with the higher bill.
- Create a `HospitalManager` class and declare it as a friend of `Patient`. It should be able to access private patient information and generate a complete patient report.

C. Operator Overloading

Overload the following operators:

1. `+` to combine the daily treatment charges of two patient records, returning the combined bill amount.
2. `>` to compare the total bills of two patients.
3. `==` to check whether two patients have equal total bills.
4. `[]` to access the treatment charge for a particular day.
5. `<<` to display the complete patient record.
6. Prefix and postfix `++` to increase the treatment charge for a particular day by a fixed amount of Rs. 100, ensuring the operation is reflected in the total bill.

D. Billing Calculation

The hospital charges Rs. 1,000 per day for regular treatment. Additional daily charges may vary depending on the treatment received.

The program should calculate the final bill using the total of all daily treatment charges. It should allow the hospital manager to compare patient bills and generate a report.

Sample Output:

```
Enter the number of patients: 2

Enter details for Patient 1:
Patient ID: 501
Name: Ahmed
Age: 45
Number of treatment days: 3

Enter daily treatment charges:
Day 1: 1000
Day 2: 1500
Day 3: 1000

Enter details for Patient 2:
Patient ID: 502
Name: Sara
Age: 32
Number of treatment days: 2

Enter daily treatment charges:
Day 1: 2000
Day 2: 1500

Patient Billing Report:

Patient 1: Ahmed
Total Bill: Rs. 3500

Patient 2: Sara
Total Bill: Rs. 3500

Comparing Patient Bills:
Both patients have equal treatment bills.

Accessing Daily Charges:
Enter patient number: 1
Enter day number: 2

Treatment Charge: Rs. 1500

Total Registered Patients: 2
```